

GWexpy – Root Cause Analysis of Continuous Integration (CI) Failures

このレポートでは、`tatsuki-washimi/gwexpy` リポジトリにおける CI 失敗の根本原因を特定し、改善策を提案します。調査は GitHub の CI ログや自動生成された失敗レポートを解析し、Notebook Tests、Tests、Documentation の各ワークフローに分けて行いました。

1. Notebook Tests ワークフローの失敗

Notebook Tests ワークフローでは、Jupyter ノートブックを `papermill` で実行し、結果を検証します。以下の原因が繰り返し報告されていました。

原因	説明	出典	推奨対策
ノートブック内のインデント不正	ノートブックのコードセルで <code>for</code> 文や <code>with</code> ブロックの後にインデントが無い、またはブロックが重複しているため <code>IndentationError: expected an indented block</code> が発生し、実行が中断していました ¹ 。	自動生成 Issue のログでは、 <code>Cell In[9]</code> など特定セルで <code>IndentationError</code> が出たことが記録されています。	ノートブックに含まれる <code>warnings.catch_warnings()</code> ブロックを整理し、ブロック全体を単一の <code>with</code> で包むよう修正する。コード実行前に <code>nbformat</code> と <code>nbconvert</code> による構文チェックを CI へ追加し、インデント不正を事前検知する。
構文エラー (位置引数の順序)	あるノートブックでは <code>plt.colorbar()</code> 呼び出しでキーワード引数の後に位置引数を指定していたため <code>SyntaxError: positional argument follows keyword argument</code> が発生しました ¹ 。	CI: Notebook Tests failed のログに <code>SyntaxError</code> と具体的な行番号が含まれています。	該当ノートブックの呼び出し順序を修正し、関数の引数を正しい順序で記述する。
未定義変数・オプション依存の不足	<code>control</code> や <code>statsmodels</code> などのパッケージがインストールされていないため、ノートブック実行時に <code>ModuleNotFoundError</code> が発生していました。これらは <code>imeinuit</code> と同じ optional dependency として <code>requirements-dev.txt</code> に含まれていませんでした。	Notebook Tests 失敗の Issue では、 <code>control</code> や <code>statsmodels</code> の import エラーが表示されています。	<code>requirements-dev.txt</code> に <code>control</code> 、 <code>statsmodels</code> 、 <code>scikit-learn</code> 、 <code>PyWavelets</code> 等の optional 依存を追加し、CI 環境でインストールする。

Notebook Tests に対する総合的な改善策

1. ノートブック修正スクリプト – `warnings.catch_warnings()` ブロックを検知し、すべてのノートブックで壊れたブロックを削除して正しい形式で包み直す自動スクリプトを作成し適用する。
2. インデント・構文チェックの CI 追加 – ノートブックを PR ごとに `nbformat.read` と `nbconvert` で検証し、構文エラーがあればテストを失敗させる。これにより、壊れたノートブックが取り込まれるのを防止できる。
3. オプション依存を明示的にインストール – `requirements-dev.txt` および CI の install ステップに `control`, `statsmodels`, `scikit-learn`, `PyWavelets` 等を追加し、ノートブックで利用可能にする。

2. Tests ワークフローの失敗

単体テスト (`pytest`) を実行するワークフローでは、主に以下の問題が見つかりました。

原因	説明	出典	推奨対策
<code>gwpy</code> バージョン変更による <code>ImportError</code>	テストコードでは <code>gwpy.table</code> モジュールの <code>iter_h5_full_names</code> 、 <code>OrderedDict</code> 、 <code>GravitySpyTable</code> など古い関数をインポートしていましたが、新しい <code>gwpy</code> バージョンではこれらが削除または移動したため <code>ImportError</code> が発生しました。	CI: Tests failed の自動 Issue に多数記録されており、 <code>ImportError: cannot import name 'GravitySpyTable'</code> などのエラーが繰り返し報告されています。	テストコードを最新の <code>gwpy</code> API に合わせて更新するか、CI で <code>gwpy</code> を互換性のあるバージョンに固定する。また、不要な依存を取り除く。
<code>TemporaryFilename</code> の削除	<code>gwpy.testing.utils</code> から <code>TemporaryFilename</code> が削除され、インポートに失敗しています。	いくつかの Tests failed Issue で <code>ImportError: cannot import name 'TemporaryFilename'</code> が報告されています。	テストコードを修正し、 <code>TemporaryFilename</code> を使用しないようにするか、適切な代替手段を採用する。
<code>iminuit</code> の欠如	<code>gwexpy/fitting/core.py</code> が <code>iminuit</code> を import していますが、CI 環境にインストールされていないため <code>ModuleNotFoundError</code> が発生しました。	初期の CI 失敗では <code>ModuleNotFoundError: No module named 'iminuit'</code> が報告されました。	<code>requirements-dev.txt</code> と CI インストール手順に <code>iminuit</code> を追加する。

原因	説明	出典	推奨対策
<code>from __future__ import annotations</code> の位置	テストモジュールで <code>from __future__ import annotations</code> がファイルの途中に記述されていたため、Python 3.7+ では <code>SyntaxError</code> となりテストが収集されませんでした。	いくつかの <code>Tests failed</code> Issue に <code>SyntaxError: from __future__ import annotations must occur at the top of the file</code> が記録されています。	すべてのテストファイルで <code>from __future__ import annotations</code> を最上部 (モジュール docstring の直後) に移動する。
その他の欠損依存	テストではオプション依存の <code>control</code> , <code>statsmodels</code> , <code>scikit-learn</code> , <code>PyWavelets</code> , <code>pmdarima</code> , <code>dcor</code> , <code>hurst</code> , <code>EMD-signal</code> 等が必要ですが、CI 環境に存在しませんでした。	<code>Tests failed</code> と <code>Notebook Tests failed</code> における <code>ImportError</code> ログから。	これらを <code>requirements-dev.txt</code> に追加し、CI でインストールする。

Tests ワークフローに対する改善策

1. `gwp` 互換性対応 – テストコードを新しい `gwp` バージョンに対応させるか、バージョンを固定する。削除された関数の代替を実装する。
2. `__future__` インポート位置の統一 – テストファイルにある `from __future__ import annotations` をトップに移動し、他のコードより前に読み込まれるように修正する。
3. 欠損依存の追加 – `iminuit` やその他の optional ライブラリを `requirements-dev.txt` に追加し、CI でインストールする。

3. Documentation ワークフローの失敗

ドキュメントビルド (`docs`) のワークフローでは、次の問題が報告されています。

原因	説明	出典	推奨対策
<code>pyqtgraph</code> モジュールが無い	ドキュメントのビルド中に <code>ModuleNotFoundError: No module named 'pyqtgraph'</code> が発生し、Sphinx が停止しています ¹ 。 <code>pyqtgraph</code> は GUI/プロット用の依存ですが <code>docs</code> 用の環境に含まれていません。	CI: Documentation failed のログ冒頭に <code>ModuleNotFoundError: No module named 'pyqtgraph'</code> が繰り返し出ています ¹ 。	<code>pyqtgraph</code> とその依存を <code>requirements-dev.txt</code> またはドキュメント専用の環境に追加する。GUI が必須でなければ、ドキュメントビルドでは <code>gwxpy.gui</code> をスキップするように Sphinx の <code>autodoc_mock_imports</code> に <code>pyqtgraph</code> を登録する。

原因	説明	出典	推奨対策
<code>AttributeError: module 'gwexpy' has no attribute 'gui'</code>	<code>gwexpy.gui</code> 属性が存在せず、ドキュメント生成でインポートに失敗しています ¹ 。	ドキュメントの失敗ログには <code>AttributeError: module 'gwexpy' has no attribute 'gui'</code> が出ています。	GUI 機能が実際には未実装であれば、ドキュメントの autodoc から <code>gwexpy.gui</code> を除外する。実装する場合はモジュールを提供する。
GWpy intersphinx inventory の 404	<code>https://gwpy.github.io/docs/stable/objects.inv</code> へのアクセスが 404 であるため、Sphinx の intersphinx が失敗しています ² 。	CI: Documentation failed のログに <code>requests.exceptions.HTTPError: 404 Client Error: Not Found for url: ...objects.inv</code> が記録されています ² 。	<code>docs/conf.py</code> の <code>intersphinx_mapping</code> で <code>gwpy</code> の参照先を <code>https://gwpy.readthedocs.io/en/stable/objects.inv</code> に更新し、失敗してもビルドを続行するよう <code>suppress_warnings</code> を設定する。
オプション依存の不足	<code>gwexpy</code> のドキュメントは <code>control</code> や <code>statsmodels</code> など optional 依存の関数を自動ドキュメント化しますが、環境に無いため <code>ImportError</code> が発生しています。	ドキュメントの一部では <code>ModuleNotFoundError: No module named 'control'</code> や <code>statsmodels</code> の import エラーが記録されています。	<code>requirements-dev.txt</code> と CI の docs 環境にこれらの依存を追加する。必要ないモジュールは <code>autodoc_mock_imports</code> に追加してモック化する。

Documentation ワークフローに対する改善策

1. 環境に必要な依存をインストール - `pyqtgraph` や `control` などドキュメント生成に必要なライブラリを dev 依存に追加し、CI でインストールする。不要な GUI モジュールは `autodoc_mock_imports` にモック化してドキュメントのみのビルドを行う。
2. Intersphinx 設定の修正 - `docs/conf.py` で `gwpy` の intersphinx マッピングを正しい URL に変更し、失敗時に警告を抑制する。
3. GUI モジュールの整理 - 未実装の `gwexpy.gui` を `__init__.py` から削除するか、Sphinx の自動ドキュメント対象から外す。

4. 総合的な推奨対策

1. CI 環境の統一と依存管理
2. 全ワークフロー (Tests, Documentation, Notebook Tests) で同じ `requirements-dev.txt` を使用し、`control`, `statsmodels`, `scikit-learn`, `PyWavelets`, `pmdarima`, `dcor`, `hurst`, `EMD-signal` 等すべての optional 依存をインストールする。
3. `pip install` と `conda/mamba` の両方で依存を追加し、環境差異を無くす。
4. コードベースの整理

5. ノートブック、テスト、ドキュメントそれぞれで不要になった古い API 呼び出し (古い `gwp` モジュール) を削除し、新しい API に対応させる。

6. `__future__` インポートの位置を統一し、テストファイルを PEP 502 に適合させる。

7. CI ログサマライザーの改良

8. `CI Log Summarizer` が bot 実行や同一エラーで大量の Issue を作成しないよう、既に導入した fingerprint に加えて actor フィルタリングや重複検出を強化する。

9. 失敗修正期間中は Summarizer を一時的に無効化し、安定後に再有効化する。

10. 運用改善

11. Dependabot の更新頻度を週次に制限し、一度に多数の依存更新 PR が開くのを防ぐ。

12. ノートブックやドキュメントの CI チェック結果をチーム内で共有し、早期に不具合を検知できるようにする。

おわりに

本分析で、CI 失敗の主な原因は **ノートブックの構文不正**、**テストコードの古い `gwp` API 依存**、**欠損依存のインストール漏れ**、および **ドキュメント用モジュールの不足や設定不良** であることが分かりました。上記の改善策を実施することで、現在の CI エラーを解消し、将来の自動化テストとドキュメント生成の信頼性を高めることができます。

¹ CI: Documentation failed (Ref: 24204872851)

<https://github.com/tatsuki-washimi/gwexpy/issues/124>

² CI: Documentation failed (Ref: 24184393333)

<https://github.com/tatsuki-washimi/gwexpy/issues/93>